

Exercise 1: Ranking and Window Functions:

Code:

```
CREATE TABLE Products (  
    ProductID INTEGER PRIMARY KEY,  
    ProductName TEXT,  
    Category TEXT,  
    Price DECIMAL(10,2)  
);  
  
INSERT INTO Products (ProductName, Category, Price) VALUES  
('Laptop Pro', 'Electronics', 1200.00),  
('Smartphone X', 'Electronics', 800.00),  
('4K Monitor', 'Electronics', 400.00),  
('Gaming Console', 'Electronics', 400.00), -- Tie price to test RANK vs DENSE_RANK  
('Bluetooth Speaker', 'Electronics', 150.00),  
('Ergonomic Chair', 'Furniture', 350.00),  
('Standing Desk', 'Furniture', 500.00),  
('Bookshelf', 'Furniture', 200.00),  
('Office Lamp', 'Furniture', 50.00),  
('Running Shoes', 'Apparel', 120.00),  
('Winter Jacket', 'Apparel', 200.00),  
('Designer Jeans', 'Apparel', 150.00),  
('T-Shirt', 'Apparel', 25.00);  
  
WITH RankedProducts AS (  
    SELECT  
        Category,  
        ProductName,  
        Price,  
        ROW_NUMBER() OVER (PARTITION BY Category ORDER BY Price DESC) AS RowNum,  
        RANK() OVER (PARTITION BY Category ORDER BY Price DESC) AS RankNum,  
        DENSE_RANK() OVER (PARTITION BY Category ORDER BY Price DESC) AS DenseRankNum  
    FROM Products
```

)

```
SELECT * FROM RankedProducts
```

```
WHERE DenseRankNum <= 3; -- Filters for top 3 per category based on dense rank
```

Output:

Exercise 1: Ranking and Window Functions:

Output					
Category	ProductName	Price	RowNum	RankNum	DenseRankNum
Apparel	Winter Jacket	200	1	1	1
Apparel	Designer Jeans	150	2	2	2
Apparel	Running Shoes	120	3	3	3
Electronics	Laptop Pro	1200	1	1	1
Electronics	Smartphone X	800	2	2	2
Electronics	4K Monitor	400	3	3	3
Electronics	Gaming Console	400	4	3	3
Furniture	Standing Desk	500	1	1	1

Exercise 2: Create a Stored Procedure

Code:

```
SQL> CREATE OR REPLACE PROCEDURE sp_GetEmployeesByDepartment
```

```
    p_DepartmentID IN NUMBER
```

```
)
```

```
AS
```

```
BEGIN
```

```
    FOR emp_rec IN (
```

```
        SELECT EmployeeID,
```

```
            FirstName,
```

```
            LastName
```

```
        FROM Employees
```

```
        WHERE DepartmentID = p_DepartmentID
```

```
)
```

```
LOOP
```

```

        DBMS_OUTPUT.PUT_LINE(
            emp_rec.EmployeeID || ' ' ||
            emp_rec.FirstName || ' ' ||
            emp_rec.LastName
        );
    END LOOP;
END;
/

```

employee table

Output:

EMPLOYEEID FIRSTNAME LASTNAME DEPARTMENTID SALARY JOINDATE

```

-----
1      Alice   Joe    11      51000 16-JAN-25
2      Joe    Smith  20      70000 19-JUN-22
3      Mike   Thomson 10      55000 20-MAY-25

```

Exercise 3: Return Data from a Stored Procedure

Code:

SQL> CREATE OR REPLACE PROCEDURE sp_GetEmployeeCount

```

    p_DepartmentID IN NUMBER,
    p_TotalEmployees OUT NUMBER
)
AS
BEGIN
    SELECT COUNT(*)
    INTO p_TotalEmployees
    FROM Employees
    WHERE DepartmentID = p_DepartmentID;
END;

```

/

Procedure created.

```
SQL> VARIABLE total_count NUMBER;
```

```
SQL> EXEC sp_GetEmployeeCount(10, :total_count);
```

PL/SQL procedure successfully completed.

```
SQL> PRINT total_count;
```

TOTAL_COUNT

1